Aran Learn 23: One UI for the Console, Aran Cloud and the Demo Banks

For: everyone who will run or change Aran Cloud (`aran_cloud/`), the demo bank (`demo_bank/`) or the shared design kit (`app/ui/kit/`). Aasif first for parts 3 and 4, Arun and Bharadhwaj for parts 1, 2 and 5. **Time:** about 2 hours with practice. **Before this:** Learn 8 (web APIs and proxies), Learn 11 (frontend basics), Learn 15 (design tokens) and Learn 17 (wiring a design to a live API).

The `aran-ui-banks-cloud` branch joins two pieces of work. Arun's `BanksAndCloud` branch added a multi-tenant **Aran Cloud**, a **demo bank** in two modes and an SDK. The team's **Aran UI** (the console in `app/ui`) had the look we wanted. Now every page in every app uses the Aran UI, and every number on those pages comes from the running servers. This doc teaches the five ideas you need: **four services, one product**, **a shared design kit**, **multi-tenant workspaces**, **API keys with scopes**, and **rate limits you can trust**.

---

1. Four services, one product

Aran now runs as four small FastAPI servers. Each has one job.

Port	Service	Folder	What it does
8000	Aran core	`app/server`	The firewall itself: detection, tokens, erasure, receipts. Also serves the console at `/` .
8100	Aran Cloud	`aran_cloud`	Workspaces, API keys, metering and dashboards in front of the core.
9001	Bank A	`demo_bank` with `BANK_MODE=legacy`	A typical bank that stores and sends raw identifiers.
9002	Bank B	`demo_bank` with `BANK_MODE=aran`	The same bank, calling Aran Cloud through the SDK.

The path of one sign-up at Bank B: browser → Bank B `/api/customers` → Aran Cloud `/v1/protect` (checks the key, adds the workspace name to the customer ref) → core `/api/process` (makes the tokens) → back to Bank B, which stores only the tokens.

Start everything with `scripts/run_demo.sh` . It prints the address of every page.

---

2. A shared design kit

Copying `style.css` into four apps would mean four copies drifting apart. Instead, Aran Cloud and the bank **serve the console's own folder** at `/aran` :

```
ARAN_UI_DIR = os.environ.get("ARAN_UI_DIR", os.path.join(HERE, "..", "app", "ui"))
app.mount("/aran", StaticFiles(directory=ARAN_UI_DIR), name="aran-ui")
```

Every page then loads the same three style sheets and two scripts:

```
<link rel="stylesheet" href="/aran/css/tokens.css">   <!-- colours, type, spacing (Learn 15) -->
<link rel="stylesheet" href="/aran/css/style.css">    <!-- cards, buttons, pills, tables, top bar -->
<link rel="stylesheet" href="/aran/kit/kit.css">      <!-- small extras: app switcher, chat, charts -->
<script src="/aran/js/ui.js"></script>                <!-- component helpers: U.stat, U.pill, U.note ... -->
<script src="/aran/kit/kit.js"></script>              <!-- shell, fetch helper, highlighting -->
```

`ARAN.kit.shell({ app: 'bankB', brand: 'Bank B' })` draws the console's top bar, the full-screen menu, the theme toggle and a row of links to every app. Change a colour in `tokens.css` and the console, the cloud and both banks change together. Theme choice is kept under the same `aran-theme` key, so light mode follows you between apps on the same host.

Try it: change `--token-color` in `app/ui/css/tokens.css`, reload Bank B, and watch every token turn the new colour.

Two small tricks worth knowing:

- **Embedded mode.** The side-by-side page shows both banks in `<iframe>`s with `?embed=1`. `kit.js` adds the `embed` class to `<html>`, and `kit.css` hides the top bar inside the frame.
- **Meaning colours.** Real data is amber, tokens are teal, leaks are red-orange (spec section 0). Bank B's Aadhaar tokens are 12 digits, just like real Aadhaar numbers (format-preserving encryption, Learn 3), so the page cannot guess from the shape. It marks digits as tokens when it knows the bank is in Aran mode: `K.hl(text, { digits: 'token' })`.

---

3. Multi-tenant workspaces

A **tenant** is one customer of a service: here, one bank. **Multi-tenant** means many banks share one Aran Cloud, and none of them may see another's data.

Aran Cloud keeps each bank apart in three ways:

1. Every table row in `cloud.db` has a `tenant_id`, and every query filters on it.
2. Customer refs are **namespaced** before they reach the core: `CUST-1001` from workspace `t_ab12` becomes `t_ab12__CUST-1001`. Two banks can both have a `CUST-1001`.
3. `/v1/trace` and `/v1/detokenise` only answer for tokens this workspace issued.

Fixed since (audit item 5): the core used to decide who "owns" an Aadhaar number globally, so if Bank B sent a number Bank A had already enrolled, the answer said "already enrolled elsewhere" and Bank B learned that the person banks with someone else. Now Aran Cloud sends its tenant id as a `scope`, and the core looks up ownership inside that tenant only. Learn 25 explains how.

---

4. API keys with scopes

An **API key** is a long random secret that a program sends in a header (`X-API-Key`) to prove who it is. Aran Cloud stores only the **SHA-256 hash** of each key. That is why a new key is shown once: the server cannot show it again.

A **scope** limits what a key can do. A workspace starts with two keys:

Key	Scopes	Used by
Admin key	all eight	the bank's back end, setup scripts
Dashboard key	`dashboard` only	the dashboard page, read-only

The dashboard page opens with the read-only key. When you press an action that needs more (erase, trace, mint or revoke a key, run demo traffic), the server answers **403 Forbidden**, and the page asks for the admin key inline instead of failing. This is **least privilege** (Learn 22): the key that sits in a browser tab can only read.

Keys handed between pages travel in the URL **fragment** (`/dashboard#key=…`), never the query string, because the fragment is not sent to the server or written to its logs. The page saves it to `sessionStorage` (this tab only) and removes it from the address bar.

---

5. Rate limits you can trust

A **rate limit** caps how often one caller may do something, for example 5 sign-ups an hour from one address. To count per address, the server needs the caller's IP.

Behind a proxy (Render, nginx, a load balancer) every request seems to come from the proxy, so proxies add an `X-Forwarded-For` header with the real address. The catch: **anyone can send that header**. The audit showed that by changing it, one person could create unlimited workspaces.

The fix in `aran_cloud/main.py`:

```
 TRUSTED_PROXIES = {p.strip() for p in os.environ.get("ARAN_TRUSTED_PROXIES",
"").split(",") if p.strip()}

def client_ip(request):
    peer = request.client.host                    # the machine that really opened the
connection
    fwd = request.headers.get("X-Forwarded-For")
```

```
    if fwd and peer in TRUSTED_PROXIES:           # only believe the header from our own
proxy
        return fwd.split(",")[0].strip()
    return peer
```

- With `ARAN_TRUSTED_PROXIES` empty (the default), the header is ignored.
- Uvicorn has its own copy of this logic: by default it believes `X-Forwarded-For` from 127.0.0.1 and rewrites the caller's address before our code sees it. So Aran Cloud is started with `--no-proxy-headers` (in `run_demo.sh` and `docker-compose.yml`), and our `client_ip` makes the only decision.
- On Render or behind nginx, set it to the proxy's address.
- Demo mode still skips the limit for a real local caller, but not for a request that only claims to be local in a header.

Also new: the core's console files (`/`, `/css`, `/js`) are always public, and `ARAN_CONSOLE_OPEN=1` lets the console's own `/api` calls through for a core that only listens on 127.0.0.1. That fixes audit item 8 for the local demo.

The second hole was `POST /v1/internal/reset-limiters`, which anyone could call to clear every limit. It now needs `ARAN_ADMIN_TOKEN` in an `X-Admin-Token` header, and is switched off when no token is set. The demo seeder (`/v1/demo/seed`) now needs a key with the `admin` scope.

Try it: start Aran Cloud with `--no-proxy-headers` and without `ARAN_TRUSTED_PROXIES`, then send six sign-ups with `curl -H "X-Forwarded-For: 10.0.0.$RANDOM"`. The sixth now gets `429 Too Many Requests`.

Exercises

1. Add a "Last used" column to Bank B's officer view that reads the Aran Cloud audit log. Which key scope does Bank B need?
2. Add an eighth link to the app switcher in `kit.js` for the console's `#versus` page.
3. Run `aran_cloud/test_cloud_security.py` twice: once with `ARAN_TRUSTED_PROXIES=127.0.0.1` and once without. Explain why the rate-limit test only passes with it.
4. Read `_owner_key` in `research/firewall/shredder.py`. Why does the empty scope keep the old key format?

Words to know

Word	Meaning
Tenant	One customer of a shared service (here, one bank).
Namespace	A prefix that keeps two tenants' names apart (`t_ab12__CUST-1001`).
Scope	A permission attached to an API key.
Least privilege	Give each key or person only the access they need.

Word	Meaning
URL fragment	The part after `#`. Browsers never send it to the server.
Trusted proxy	A proxy you run, whose `X-Forwarded-For` header you believe.
Rate limit	A cap on requests per caller per time window; over it, the server answers 429.
Static mount	Serving a whole folder of files under one URL path.